U.S. Government Financial Report - Multi-Modal Agentic RAG

System Setup, Architecture, Autonomous Agent Roles, Deployment Steps, Challenges & Limitations

Author: Rama Kotipatruni | Dataset: U.S. Treasury Statements of Net Cost (FY2022-FY2025: XML, JSON, CSV)

1. Executive Summary & Project Overview

The U.S. Government Financial Report (USFR) Multi-Modal Agentic RAG System is an autonomous knowledge and decision-support application for enterprise and federal financial intelligence. It enables analysts, auditors, policymakers, and citizens to upload and query heterogeneous enterprise and federal financial documents (XML, JSON, CSV, Excel, PDF, TXT, and Markdown) using natural language.

Traditional Retrieval-Augmented Generation (RAG) systems struggle with financial statements because character-level text splitting severs numbers from their agency and fiscal-year labels, and Large Language Models (LLMs) are prone to arithmetic hallucinations when computing rankings or year-over-year changes. This system solves both problems by combining multi-format schema normalization, four-level semantic chunking in ChromaDB, deterministic pandas analytical tools, and a 6-node LangGraph multi-agent workflow with an automated numeric grounding validator.

Capstone Task	Requirement Summary	Implementation Module(s)
1. Project Foundation	Repository layout, .env config, modular structure	src/config.py, setup_key.py, .env.example
2. User Interaction Layer	Web UI + CLI for uploads and natural-language Q&A	app.py (Streamlit 3-tab UI), main.py (REPL)
3. Document Ingestion	Multi-format loading (XML, JSON, CSV, XLSX, PDF, TXT)	src/ingestion/loaders.py, ingest.py
4. Semantic Chunking	Multi-granularity financial chunks + recursive text split	src/ingestion/documents.py (295 chunks)
5. Vector Knowledge Store	Local embeddings + persistent vector DB	src/vectorstore.py (ChromaDB + bge-small)
6. Intelligent Retrieval	Hybrid similarity search + structured analytical tools	src/agents/tools.py, src/data_store.py
7. Grounded RAG Pipeline	Numbered context [S1..Sn] + strict inline citations	src/agents/graph.py (reasoner node)
8. Agent-Based Reasoning	Autonomous planning, tool execution, validation loop	src/agents/graph.py (6-node LangGraph)
9. Safety & Reliability	Prompt-injection/PII guardrails + numeric verification	src/guardrails.py, validator node
10. Deployment & Docs	Streamlit deployment, architecture diagrams & tests	docs/ARCHITECTURE.md, tests/test_system.py

2. System Setup & Environment Configuration

2.1 Technology Stack & Open-Source Dependencies

- **Agent Orchestration:** LangGraph (>=0.2.0) and LangChain Core/Community (>=0.3.0) for state-machine agent workflows and tool binding.
- **LLM Providers:** Google Gemini ('gemini-3-flash-preview' with automatic fallback to 'gemini-2.5-flash-lite', 'gemini-3.1-flash-lite', and 'gemini-2.5-pro') for cloud inference, or Ollama ('llama3.1:8b') for 100% local open-source execution.

- **Local Embeddings & Vector DB:** HuggingFace 'BAAI/bge-small-en-v1.5' (sentence-transformers, CPU-friendly, cosine-normalized) persisted in ChromaDB ('chroma_store/').
- **Multi-Format Parsing & UI:** pandas, openpyxl, ElementTree, and pypdf for ingestion; Streamlit and Plotly for the interactive web dashboard.

2.2 Step-by-Step Installation & API Key Configuration

Terminal Setup & Ingestion Commands

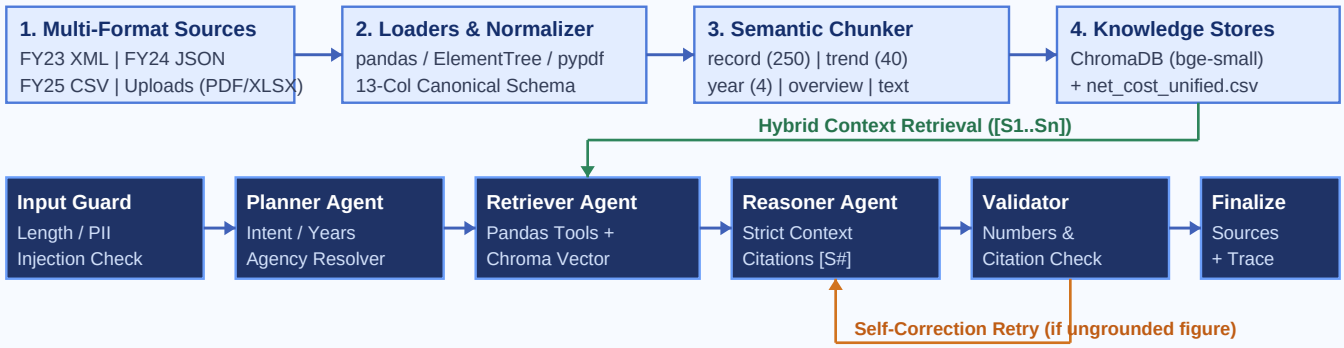
```
# 1. Create and activate Python 3.11+ virtual environment
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# 2. Configure API key in .env (via interactive CLI, flag, or Streamlit sidebar)
python setup_key.py --key AIzaSyYourGeminiKeyHere
# Or switch to 100% local open-source Ollama:
python setup_key.py --provider ollama --model llama3.1:8b

# 3. Build / refresh the unified structured table and ChromaDB vector store
python ingest.py --reset
```

3. System Architecture & Data Pipeline

Figure 1: End-to-End Multi-Modal Ingestion & 6-Node LangGraph Agentic RAG Workflow



3.1 Multi-Format Ingestion & Canonical Schema Normalization

The project ships with three official U.S. Treasury Fiscal Data files spanning FY2022-FY2025 in three different formats: 'USFR_StmtNetCost_20230101_20231231.xml' (FY2023 + FY2022 restated), 'USFR_StmtNetCost_20240101_20241231.json' (FY2024 + FY2023 restated), and 'USFR_StmtNetCost_20250101_20250930.csv' (FY2025 + FY2024 restated). In addition, users can upload custom PDF, TXT, Markdown, CSV, Excel (.xlsx/.xls), JSON, and XML files up to 25 MB.

In 'src/ingestion/loaders.py', FIELD_MAP maps both API field names (such as 'stmt_fiscal_year', 'gross_cost_bil_amt', 'restmt_flag') and human-readable CSV headers ('Statement Fiscal Year', 'Gross Cost (in Billions)', 'Restatement Flag') into a unified 13-column canonical table ('data/processed/net_cost_unified.csv', 250 deduplicated rows across 40 federal entities). All numeric columns are rounded to 1 decimal place to eliminate IEEE-754 floating-point artifacts.

3.2 Four-Level Semantic Chunking Strategy

Rather than splitting financial tables by arbitrary character counts, 'src/ingestion/documents.py' synthesizes self-contained natural-language chunks at four complementary granularities, each tagged with rich metadata and a deterministic SHA-1 ID for idempotent ChromaDB upserts:

- **1. Record Chunks (250 chunks):** One natural-language block per (agency, statement FY, report FY) row containing gross cost, earned revenue, subtotal, assumption changes, net cost, restatement status, and source file/line lineage.
- **2. Agency Trend Chunks (40 chunks):** One multi-year longitudinal summary per agency across FY2022-FY2025 with pre-computed year-over-year dollar and percentage deltas.
- **3. Year Summary Chunks (4 chunks):** One macro summary per fiscal year (FY2022-FY2025) listing government-wide totals, top 10 highest-cost agencies, and entities with negative net cost.
- **4. Dataset Overview & Free-Text Chunks:** A domain glossary explaining federal accounting terms (Gross Cost, Earned Revenue, Changes in Assumptions, Restatement Flag) plus RecursiveCharacterTextSplitter chunks (900 chars, 150 overlap) for uploaded PDFs/TXT.

4. Autonomous Agent Roles (LangGraph Workflow)

The core reasoning engine in 'src/agents/graph.py' is implemented as a compiled LangGraph StateGraph with six specialized nodes and conditional edges:

Agent / Node	Primary Responsibility	Tools / Mechanisms Used
1. Input Guard (input_guard)	Pre-LLM security and input validation gatekeeper. Blocks malicious or invalid inputs immediately.	Regex checks for prompt injection ('ignore previous instructions', 'jailbreak'), unsafe queries, length (>1000 chars), and PII (SSN, credit card).
2. Planner Agent (planner)	Rewrites follow-up queries using chat history, classifies intent, resolves agency abbreviations, and plans tool calls.	LLM structured output (Plan Pydantic schema) + 35+ agency alias map (DoD, HHS, SSA, VA, OPM) + deterministic _heuristic_plan() fallback.
3. Retriever Agent (retriever)	Executes hybrid retrieval according to the structured Plan and formats numbered context blocks [S1..Sn].	lookup_financials, rank_agencies, compare_fiscal_years, restatement_differences (pandas) + semantic_search (ChromaDB cosine similarity).
4. Reasoner Agent (reasoner)	Synthesizes a concise, grounded markdown response strictly from [S1..Sn] at temperature=0.	Strict system prompt requiring [S#] citations on every factual claim and prohibiting unverified mental arithmetic.
5. Validator Agent (validator)	Deterministically checks numeric accuracy and citation integrity; triggers self-correction retry if needed.	ungrounded_numbers() (parses numbers, scales trillions<->billions, checks +/-0.05B tolerance) + has_citations() ([S#] existence check).
6. Finalize Node (finalize)	Packages verified answer, attaches cited source metadata, and records full execution trace for the UI.	Filters context blocks to cited [S#] IDs; appends verification note if retry budget (MAX_VALIDATION_RETRIES=1) is exhausted.

5. Deployment Steps & Verification Suite

5.1 Running Locally (Streamlit Web UI & CLI REPL)

The application supports both a browser-based multi-tab Streamlit dashboard ('app.py') and an interactive command-line REPL ('main.py'). On first launch, 'ensure_knowledge_base()' automatically bootstraps 'data/raw/' and populates 'chroma_store/' and 'data/processed/net_cost_unified.csv'.

Application Execution & Testing Commands

```
# Launch the Streamlit Web UI (http://localhost:8501)
.venv/bin/streamlit run app.py

# Launch the Interactive Terminal REPL
.venv/bin/python main.py

# Run the Automated Pytest Verification Suite (10 tests)
.venv/bin/pytest -v
```

5.2 Cloud & Container Deployment

- **Streamlit Community Cloud:** Push repository to GitHub ('.env' and 'chroma_store/' are excluded via '.gitignore'). Set 'app.py' as the entrypoint and add 'GEMINI_API_KEY' in App Settings -> Secrets. The app auto-ingests the USFR dataset on cold start.
- **100% Local / Air-Gapped Deployment:** Install Ollama, pull 'llama3.1:8b', and set 'LLM_PROVIDER=ollama' in '.env'. Combined with local HuggingFace 'BAAI/bge-small-en-v1.5' embeddings and ChromaDB, the entire stack runs offline with zero external API calls.

6. Challenges Faced & System Limitations

6.1 Key Engineering Challenges & How They Were Solved

Challenge Encountered	Root Cause	Engineering Solution Implemented
1. Heterogeneous Multi-Year File Formats	FY2023 data arrives as XML with API tags, FY2024 as nested JSON, and FY2025 as CSV with display headers.	Built unified readers in loaders.py and FIELD_MAP to normalize all three formats into a 13-column canonical table.
2. Floating-Point Artifacts in Raw Data	XML and JSON files contained IEEE-754 artifacts such as 1.39999999999999 instead of 1.4 billion.	Applied pd.to_numeric(...).round(1) during normalization so all figures match official published Treasury tables.
3. Prior-Year Restatement Double-Counting	Each annual report includes restated prior-year figures (restatement_flag='Y') alongside current-year rows ('N').	Implemented FinancialDataStore.preferred() to default to original figures ('N') while exposing restatement_diff() for revision queries.
4. LLM Arithmetic & Ranking Hallucinations	LLMs frequently miscalculate multi-row rankings, sums, or percentage changes over raw text chunks.	Offloaded all lookups, rankings, and YoY deltas to deterministic pandas tools and added ungrounded_numbers() validation.
5. Model Deprecation & API Resilience	Older model aliases ('gemini-2.5-flash') return 404 NOT_FOUND for newly issued Gemini API keys.	Updated default to 'gemini-3-flash-preview' with automatic multi-model fallbacks in src/llm.py plus heuristic planner fallback.

6.2 Current Limitations & Future Roadmap

- **Scanned Image-Only PDFs:** The PDF loader ('pypdf') extracts digital text streams. Scanned image-only PDFs without an embedded text layer require an OCR pre-processing stage (e.g., Tesseract or Google Document AI).
- **Pre-Loaded Financial Statement Scope:** The default Treasury dataset covers the Statements of Net Cost (FY2022-FY2025). Answering questions about federal tax revenue collections or debt balance sheets requires uploading the corresponding Treasury files via the sidebar uploader.
- **Single-Pass Validator Retry Cap:** To maintain sub-10-second response latency, automatic self-correction between the Validator and Reasoner is capped at MAX_VALIDATION_RETRIES=1; if still unverified, a transparent warning banner is attached.